

---

# Deep Learning Project: Online Handwritten Mathematical Expression Recognition

---

Anthony Smaldore 23225831

## Abstract

The overall objectives were to implement different types of Deep Learning models to process and classify individual handwritten glyph strokes and sequences of strokes. The Convolutional Neural Network (CNN) was implemented with five convolutional layers and two linear layers combined with specific techniques and hyperparameters that are discussed in detail in [Part 1 – CNN for Glyph Classification](#). The final classification performance achieved on the evaluation set of the dataset with 80,000 examples was 90.07%. The Recurrent Neural Network (RNN) was implemented with an encoder-decoder structure using the Gated Recurrent Unit (GRU) with a reset and update gate for both the encoder and the decoder. In addition, a lightweight CNN pre-encoder with two convolutional layers and one hidden layer was used to pre-encode the individual glyph strokes for the encoder of the RNN. Further details discussed in depth in [Part 2 – RNN for Infix Expression Recognition](#). The model recorded a Levenshtein accuracy of 95.92% and a forced decoding Character Error Rate of 3.19%. The Transformer was implemented with a decoder architecture using multi-head self-attention, and a CNN pre-encoder. There are four heads used for self-attention and the transformer decoder is built with pre-layer normalization. The CNN pre-encoder has two convolutional layers and one hidden layer that was used to pre-encode the strokes for the Transformer decoder. Further details discussed in depth in [Part 3 – Transformer for Postfix Expression Recognition](#).

## Introduction and Background

### Handwritten Recognition Approaches

The input for each of the three models are based off of online handwritten glyphs representing mathematical tokens. According to the research paper *Computer Recognition of Unconstrained Handwritten Numerals*, there are two general approaches to recognition including global analysis and structural analysis [[Suen et al., 1992](#)]. The structural analysis technique of capturing the shape features, such as the stroke from the characters, is relevant for this project. There is a fundamental difference between online handwritten recognition versus offline handwritten in terms of its representation: handwriting offline is digitally scanned and saved as image compared to handwriting online which captures the handwriting through a sensor on a digital device [[Ghosh et al., 2022](#)].

### CNNs for Classification

CNNs prove capable of completing classification tasks because of their ability to process data that is formatted with a grid like topology such as the stroke based handwritten characters. The CNN is made up of an architecture that can consist of convolution layers which use a kernel occupying a fixed set of space per layer to produce output activation maps. Crucially, these kernels preserve the spatial structure of the data through parameter sharing. The activation maps are processed through an activation function like the Rectified Linear Unit (ReLU) to compute the activations for the respective neurons. Additionally, the CNN can contain a pooling layer to allow the learned features from a convolutional layer or layers to be reduced resulting in the reduction of the overall dimensions of learned features as mentioned in the module's lecture eight. A known limitation of the CNN is its inability to represent the sequential data or data in a specific state as it is a feed-forward network that does not contain any memory of the past.

### RNNs for Sequence Modelling

RNNs address a key limitation of feed-forward networks such as the CNN, which is that they are stateful models that can handle sequential data. This ability comes from the RNN's characteristics of

shared weights across all time steps and internal memory. This allows RNNs to learn the context of general rules such as updating its internal state and making predictions based on its knowledge of an entire sequence that has been generated up to a point in time. Sequence-to-sequence models are architectures that map one sequence of data to another sequence of data and importantly are dependent on knowing where the sequence begins and ends. These sequence-to-sequence models can be represented as a many-to-one and one-to-many architectural approach through an encoder and decoder. The encoder encodes a single vector that represents the input sequence as a hidden latent state. The decoder then decodes this hidden latent state to produce the target output sequence[[Cho et al., 2014](#)]. A limitation of RNNs is that they suffer from long range memory due to vanishing or exploding gradients. The Long-Short-Term-Memory (LSTM) and Gated Recurrent Units (GRU) are modern RNN approaches that attempt to address these limitations.

### Transformers for Sequence-to-Sequence Tasks

Transformers are an upgrade to RNNs for sequence-to-sequence tasks because of their ability to use attention mechanisms[[Vaswani et al., 2017](#)]. The ability of Transformers to use attention allows for the processing of entire sequences simultaneously which provides benefits like handling memory inter dependencies at longer ranges, and the ability to capture spatial and temporal relationships like that of the handwritten strokes input. Self attention allows the network to learn linear projections to create query, key, and value representations and the computation of the attention weights is done by computing similarities between queries and keys with a scaled dot-product. Positional encoding creates the positional embeddings that provide the model with an understanding of the sequential order of the data. With the utilization of multi-head attention, multiple attention heads allows the transformer to process different represented subspaces at the same time[[Vaswani et al., 2017](#)]. One limitation of transformers is that they require massive amounts of training data and compute power.

### Experimental Setup

#### Datasets and Preprocessing

The datasets for the models were broken into a small and large dataset per model with different purposes:

Table 1: Datasets Description

| Model Name           | Small Dataset Size and Purpose | Large Dataset Size and Purpose |
|----------------------|--------------------------------|--------------------------------|
| Part 1 (CNN)         | ~ 1k; Debugging / Testing      | ~ 80k; Training                |
| Part 2 (RNN)         | ~ 1k; Debugging / Testing      | ~ 88k; Training                |
| Part 3 (Transformer) | ~ 1k; Debugging / Testing      | ~ 208k; Training               |

The online handwritten glyphs are represented with three strokes, one hundred and twenty eight features, plus an additional two features for all models. Pre-processing for all three models included removing the <bos>, <eos>, and <pad> values from the input sequences. For the RNN and Transformer, the target sequences had the <bos> and <eos> tokens added to indicate the start and end of the target token sequence. Input to all three models required padding. For the CNN, the stroke sequence was padded to the fixed length of the sequence (386). For the RNN and Transformer, the input strokes were padded with values of zero to size of the longest input sequence in the batch. The target token sequences were padded with the <pad> token to the size of the longest token sequence in the batch.

#### Training Setup

The training functionality for all three models followed a consistent pattern using two helper functions for training a single epoch and validating a single epoch for modularity, with the main training function utilizing these to implement the tracking of accuracy and loss for training and validation. The best model is saved in a checkpoint based on the validation performance. Specifically for [Part 1 – CNN for Glyph Classification](#), a confusion matrix is calculated. The below table provides an overview of the loss function, optimizer, learning rate, batch size, and more choices made for each of the models which are used during training.

Table 2: Training Setup

| Setup Choices         | Part 1 (CNN)                        | Part 2 (RNN)                                 | Part 3 (Transformer)                         |
|-----------------------|-------------------------------------|----------------------------------------------|----------------------------------------------|
| Loss Functions        | Cross-entropy                       | Cross-entropy with the padded values skipped | Cross-entropy with the padded values skipped |
| Optimizer             | Adaptive movement estimation (Adam) | Adam                                         | Adam                                         |
| Learning Rate         | $1e^{-4}$                           | $1e^{-3}$                                    | $1e^{-3}$                                    |
| Batch Size            | 128                                 | 128                                          | 128                                          |
| Teacher forcing ratio | N/A                                 | 0.0                                          | N/A                                          |
| Gradient clipping     | N/A                                 | 1.0                                          | 1.0                                          |
| Number of Epochs      | 18                                  | 18                                           | 18                                           |
| Hardware              | CPU, 6-core intel i7, Mac           | CPU, 6-core intel i7, Mac                    | GPU, T4 Google Colab                         |

## Part 1 – CNN for Glyph Classification

### Model Architecture

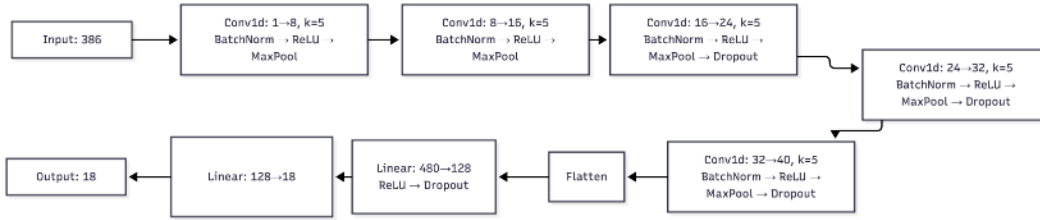


Figure 1: CNN Baseline Model

**Number of trainable parameters:** 77,090

### Hyperparameter Search

| Hyperparameter            | Search Range | Best Value                 |
|---------------------------|--------------|----------------------------|
| Number of Conv1d layers   | 1-5          | 5                          |
| Number of Conv1d channels | 1-64         | 1-40; 8 channels per layer |
| Kernel size               | 5            | 5                          |
| Dropout rate (Conv1d)     | 0.0 - 0.3    | 0.1                        |
| Dropout rate (Linear)     | 0.0 - 0.3    | 0.1                        |

### Baseline and Additional Models

The baseline model approach for the CNN was built with five convolutional layers to process the input tensor containing the features for the individual glyph strokes. The model was initialized using He Initialization for the starting weight matrices which proved better performance than random initialization. Each convolutional layer applied techniques after the pass through the convolutional layer beginning with batch normalization, ReLU for activation map computations, and max pooling for down sampling of the activation maps. For the convolutional layers three, four, and five, dropout equal to 0.1 was applied to reduce the chance of the model overfitting. The output tensor of the fifth convolutional layer is then flattened to be passed through one linear layer with its resulting output tensor size of 128 used to compute another set of activations using the ReLU activation function. Finally, dropout equal to 0.1 is applied and the tensor is passed through a second linear layer. The resulting output is of the size of the output classes for classification which is 18, and contains the final predictions for each class for the respective forward pass through the model for the input stroke sequence.

The comparison model followed a similar approach to the baseline including the same five convolutional layers using a stride of two for each convolutional layer instead of using max pooling for down

sampling of the activation maps. He Initialization of the weights was also used for the comparison model. The comparison model transposes the input tensor before and after layer normalization, and applies the Gaussian Error Linear Unit (GeLU) activation function for each convolutional layer. The third, fourth, and fifth convolutional layers apply dropout equal to 0.2 to reduce the chance of the model overfitting. The output tensor of the fifth convolutional layer is then flattened through reshaping the tensor to be passed through one linear layer with the output tensor size of 128 used to compute another set of activations using the GeLU activation function. Dropout equal to 0.3 is applied and the tensor is passed through a second linear layer with the output tensor resulting in a size of 18, and contains the final predictions for each class for the respective forward pass through the model for the input stroke sequence.

| Model      | Parameters | Val Accuracy (%) | Test Accuracy (%) |
|------------|------------|------------------|-------------------|
| Baseline   | 77,090     | 91%              | 92%               |
| Comparison | 82,000     | 92%              | 92%               |

### Training Curves

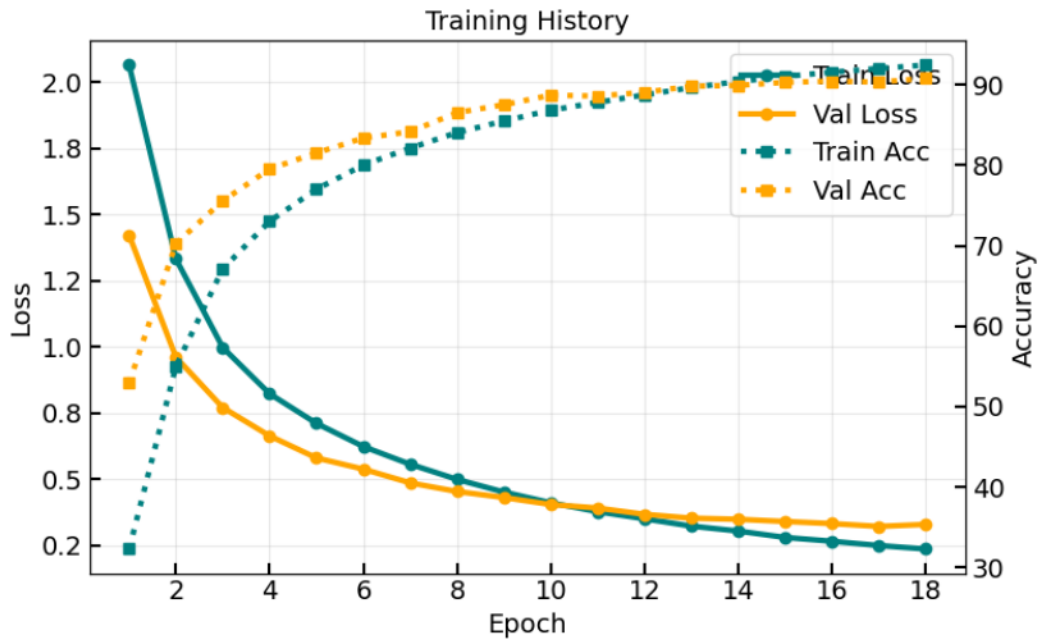


Figure 2: CNN (Baseline) Training Curve

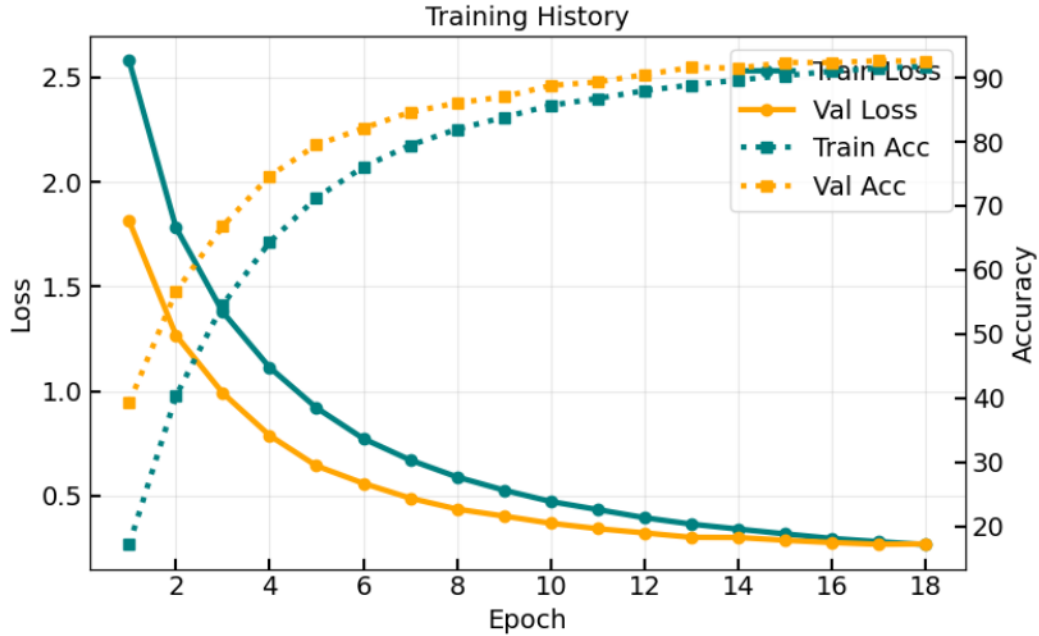


Figure 3: CNN (Comparison) Training Curve

#### Confusion Matrix CNN Baseline Model

| Class | TP   | FP  | TN    | FN  | Precision (%) | Recall (%) | F1-Score (%) |
|-------|------|-----|-------|-----|---------------|------------|--------------|
| 0     | 1527 | 139 | 30073 | 261 | 91.66         | 85.4       | 88.42        |
| 1     | 1315 | 91  | 30087 | 507 | 93.53         | 72.17      | 81.47        |
| 2     | 1640 | 43  | 30261 | 56  | 97.45         | 96.7       | 97.07        |
| 3     | 1691 | 398 | 29792 | 119 | 80.95         | 93.43      | 86.74        |
| 4     | 1723 | 240 | 29930 | 107 | 87.77         | 94.15      | 90.85        |
| 5     | 1682 | 163 | 30051 | 104 | 91.17         | 94.18      | 92.65        |
| 6     | 1623 | 109 | 30109 | 159 | 93.71         | 91.08      | 92.37        |
| 7     | 1524 | 380 | 29864 | 232 | 80.04         | 86.79      | 83.28        |
| 8     | 1653 | 164 | 30040 | 143 | 90.97         | 92.04      | 91.5         |
| 9     | 1698 | 291 | 29925 | 86  | 85.37         | 95.18      | 90.01        |
| +     | 1645 | 190 | 30076 | 89  | 89.65         | 94.87      | 92.18        |
| -     | 1641 | 78  | 30118 | 163 | 95.46         | 90.96      | 93.16        |
| *     | 1503 | 77  | 30133 | 287 | 95.13         | 83.97      | 89.2         |
| /     | 1746 | 88  | 30064 | 102 | 95.2          | 94.48      | 94.84        |
| .     | 1750 | 174 | 30076 | 0   | 90.96         | 100        | 95.26        |
| (     | 1531 | 114 | 30224 | 131 | 93.07         | 92.12      | 92.59        |
| )     | 1527 | 69  | 30091 | 313 | 95.68         | 82.99      | 88.88        |
| =     | 1631 | 142 | 30136 | 91  | 91.99         | 94.72      | 93.33        |

#### Final Performance

The final performance of the test classification accuracy for the CNN baseline model was 92%. The model improved significantly and steadily for both the training and validation accuracy over the first six epochs, then began to slow down its improvement with the following twelve epochs. Around epoch fourteen the model began to plateau around 90% and 91% for both training and validation accuracy, and the model received little improvement in accuracy over the following epochs. This indicates that the model had converged and learned the representation of features available to the model. At this point, the continuation of any training would result in the model overfitting, and signs of the model beginning to overfit are present in the final epochs as the training loss continued to drop at a higher rate than the validation loss.

## Part 2 – RNN for Infix Expression Recognition

### Model Architecture

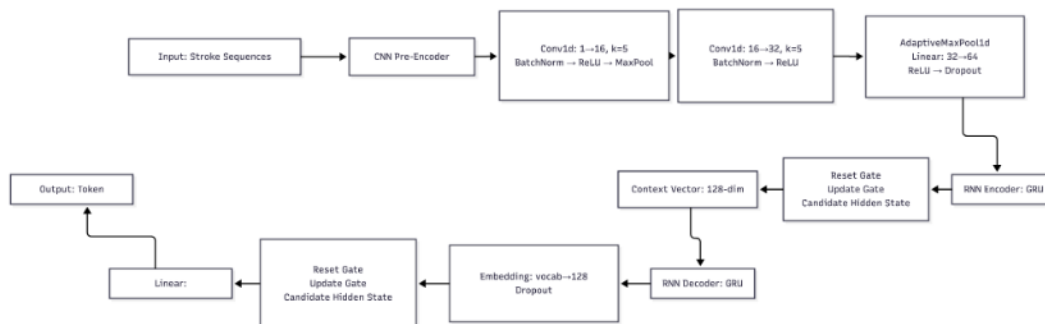


Figure 4: RNN Baseline Model

Number of trainable parameters: 210,806

### Hyperparameter Search

Table 6: Hyperparameter search

| Hyperparameter          | Search Range | CNN<br>Pre-Encoder | RNN Encoder | RNN Decoder |
|-------------------------|--------------|--------------------|-------------|-------------|
| Hidden dimension        | 64-128       | 64                 | 128         | 128         |
| Embedding dimension     | 128          | N/A                | N/A         | 128         |
| Number of RNN layers    | 1            | N/A                | 1           | 1           |
| Dropout rate            | 0.1-0.4      | 0.2                | N/A         | 0.2         |
| Number of Conv1d layers | 1-3          | 2                  | N/A         | N/A         |
| Kernel size (Conv1d)    | 5            | 5                  | N/A         | N/A         |
| Conv1d channels         | 1-64         | 1-32               | N/A         | N/A         |

### Baseline and Additional Models

The baseline model for the RNN is comprised of three parts: a CNN pre-encoder, RNN encoder, and RNN decoder. The CNN pre-encoder initially takes the input strokes tensor and adds a channel dimension to be compatible with the Conv1d layers. The input tensor is passed through the first convolutional layer applying batch normalization, then activation computation with ReLU, and is down sampled using max pooling. The tensor is then passed through a second convolutional layer applying batch normalization and ReLU for activations, and is pooled using adaptive max pooling to reduce the output to the expected fixed size of 1 for the linear layer. The linear layer processes the tensor, applies ReLU, and adds dropout equal to 0.2 to reduce the model's chances of overfitting.

The output from the pre-encoder is then flattened and passed to the GRU RNN encoder. At each time step, the encoder uses the sigmoid activation function to compute the values of the reset and update gates to force the values between 0 and 1 to determine the off or on state of these gates in the encoder. A candidate hidden state is then computed using the tanh function. The next hidden state is calculated using the previous hidden state and the candidate state based on the value of the update gate. The hidden state is updated for valid time steps including the previous hidden state for positions where there were padding.

The RNN decoder takes the output of the context vector from the RNN encoder and uses it as an initial hidden state. The weights were initialized using He Initialization. For each token to be generated in the target sequence, the input token is embedded and dropout equal to 0.2 is applied and passed

through the reset, update, and candidate state gates of the decoder to compute the new hidden state. The hidden state is then mapped to the predictions of the output classes for each of the tokens in the predicted sequence. If teacher forcing is used during training, the ground truth token will be used as input for the next time step, otherwise the model's own best prediction will be used.

The comparison RNN model followed a similar approach with two distinct differences. The CNN stroke pre-encoder was not used, and instead of the manual GRU gate implementations the advanced module from PyTorch, `nn.GRU` was used. This resulted in a much simpler architecture where an encode method applied the GRU from PyTorch implementation as opposed to the class that manually defined the encoder GRU in the baseline model. Otherwise, the structure of the model remained consistent with that was described in the baseline model.

| Model      | Parameters | Val Accuracy (%) | Test LA (%) | Test CER (%) |
|------------|------------|------------------|-------------|--------------|
| Baseline   | 210,806    | 96%              | 96%         | 3.19 %       |
| Comparison | 203,798    | 82%              | 84%         | 13.88 %      |

## Training Curves

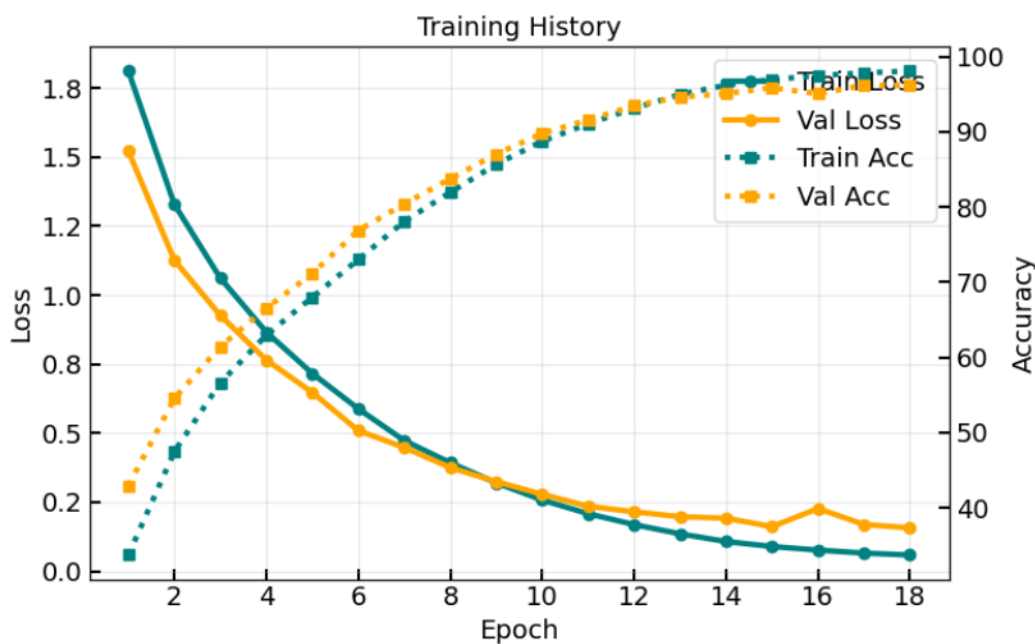


Figure 5: RNN (Baseline) Training Curve

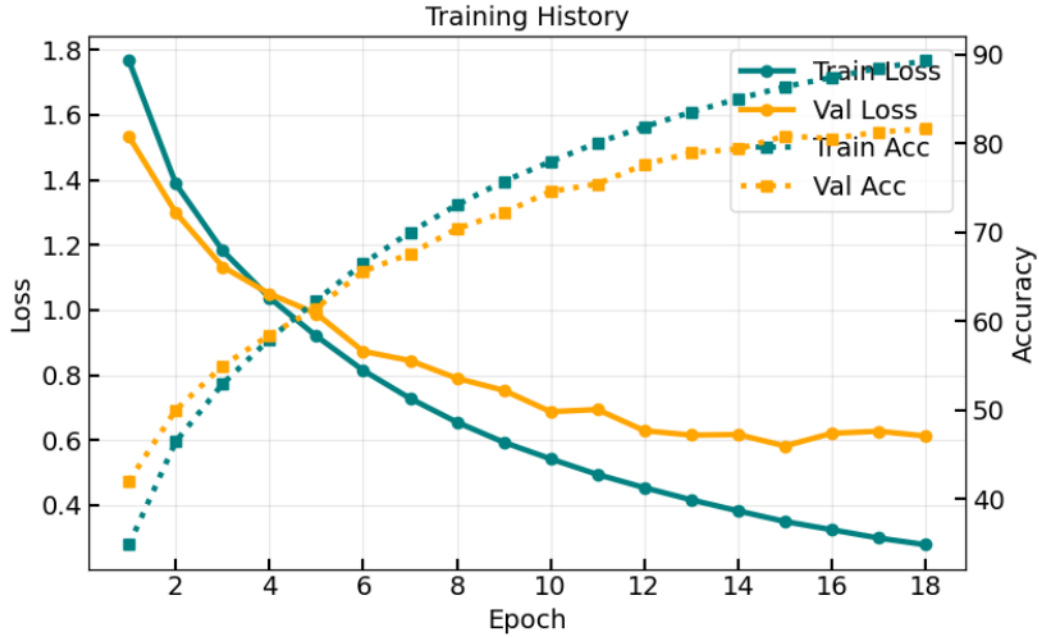


Figure 6: RNN (Comparison) Training Curve

### Final Performance

The final performance of the test Levenshtein Accuracy (LA) and forced Character Error Rate (CER) for the RNN baseline model was 96% and 3.19%, respectively. During training, the model steadily increased its training and validation accuracy across the initial ten epochs. The remaining eight epochs received a much smaller increase in accuracy for both test and validation, and around epoch fourteen appeared to have converged and learned the representation of features available to the model. This indicated that further training of model would result in overfitting after eighteen epochs.

## Part 3 – Transformer for Postfix Expression Recognition

### Model Architecture

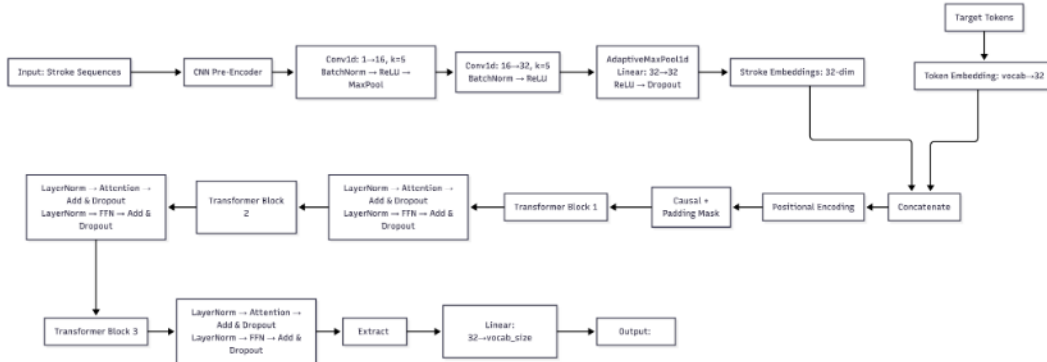


Figure 7: Transformer Baseline Model

Number of trainable parameters: 49,015

### Hyperparameter Search

Table 8: Hyperparameter search

| Hyperparameter               | Search Range | CNN Pre-Encoder | Transformer Decoder |
|------------------------------|--------------|-----------------|---------------------|
| Hidden dimension             | 32-128       | 32              | 32                  |
| Embedding dimension          | 32-128       | N/A             | 32                  |
| Number of attention heads    | 2-4          | N/A             | 4                   |
| Number of transformer blocks | 1-3          | N/A             | 3                   |
| Dropout rate (attention)     | 0.1-0.4      | N/A             | 0.3                 |
| Dropout rate (FFN)           | 0.1-0.4      | N/A             | 0.3                 |
| Dropout rate (CNN)           | 0.1-0.4      | 0.2             | N/A                 |
| Number of Conv1d layers      | 1-3          | 2               | N/A                 |
| Kernel size (Conv1d)         | 5            | 5               | N/A                 |
| Conv1d channels              | 1-64         | 1-32            | N/A                 |

### Baseline and Additional Models

The baseline model for the Transformer is comprised of two parts: the CNN pre-encoder and the Transformer decoder. The CNN pre-encoder takes the input strokes tensor and reshapes it to process all strokes across the samples simultaneously. A channel dimension is added to be compatible with the Conv1d layers. The input tensor is passed through the first convolutional layer applying batch normalization, then activation computation with ReLU, and is down sampled using max pooling. The tensor is then passed through a second convolutional layer applying batch normalization and ReLU for activations, and is pooled using adaptive max pooling to reduce the output to the expected fixed size of 1 for the linear layer. The linear layer processes the tensor, applies ReLU, and adds dropout equal to 0.2 to reduce the model’s chances of overfitting. The output is reshaped to preserve the sequence structure of the original strokes input.

The output from the CNN pre-encoder is then concatenated with an embedding of the target tokens into a single sequence with positional encodings added. A casual mask is created with the stroke and token padding masks to prevent the model from looking at future tokens or padding tokens. This masked sequence is passed to three consecutive pre-normalized transformer blocks which apply layer normalization before the multi-head self-attention and feed forward network of each block. Dropout is applied followed by linear layers. The target features are extracted and passed through a linear layer to produce the output logits for each target position in the sequence.

The comparison model follows a similar approach using the CNN pre-encoder to setup the strokes input. Positional encodings are added to the target tokens and the strokes used as memory. However, the comparison Transformer model using the built in PyTorch Transformer Decoder with three layers to perform masked self attention only on the target tokens and uses cross-attention with the target tokens as queries to look at the pre-encoded strokes. This serves as the memory and provides the keys and values.

| Model      | Parameters | Val Accuracy (%) | Test LA (%) | Test CER (%) |
|------------|------------|------------------|-------------|--------------|
| Baseline   | 49,015     | 96%              | 55%         | 14%          |
| Comparison | 61,879     | 95%              | 93%         | 5%           |

## Training Curves

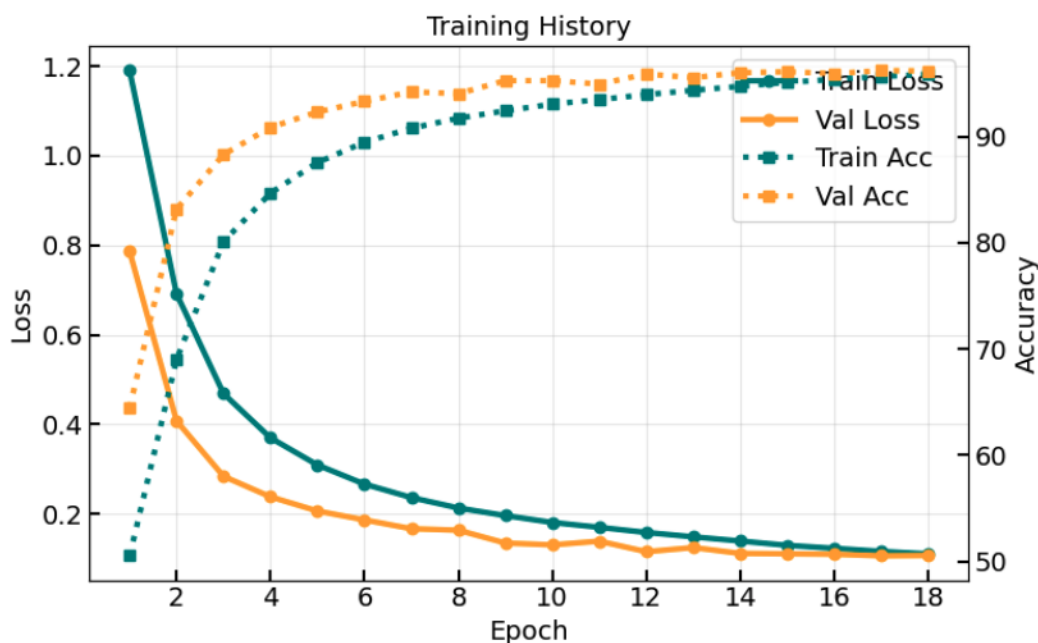


Figure 8: Transformer (Baseline) Training Curve

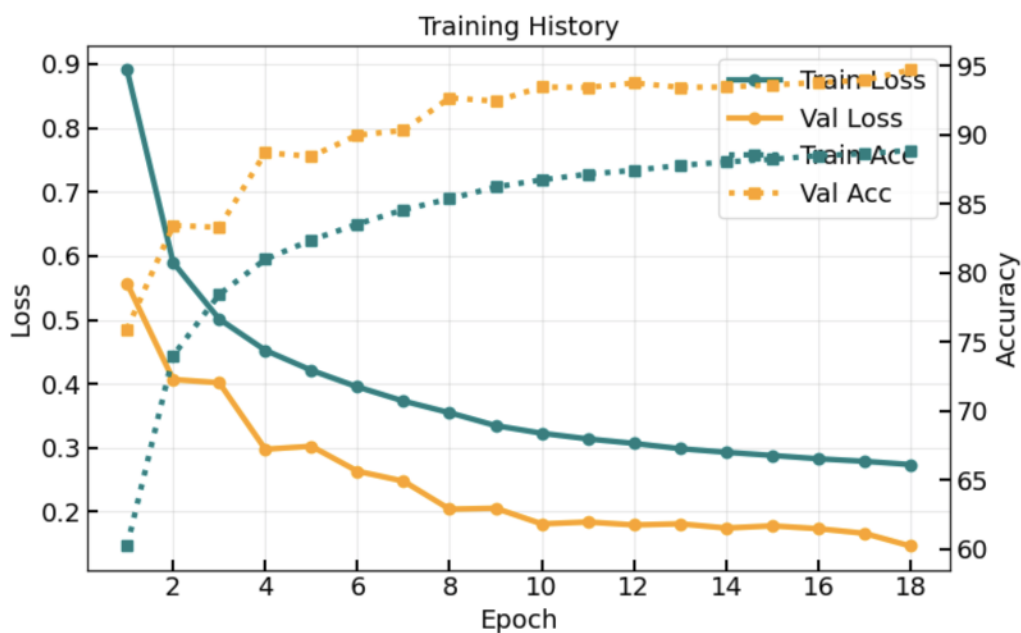


Figure 9: Transformer (Comparison) Training Curve

## Final Performance

The final performance of the test Levenshtein Accuracy (LA) and forced Character Error Rate (CER) for the Transformer baseline model was 55% and 14%, respectively. During training, the model steadily increased its training and validation accuracy across the initial 8 epochs. The remaining 10 epochs received a much smaller increase in accuracy for both test and validation, and around epoch eleven appeared to have converged and learned the representation of features available to the model.

This indicated that further training of model resulted in overfitting and the model's generalization ability suffered as seen in the final model performance on LA and forced CER.

### **Evaluation and Achievements**

Overall the model implementations for the CNN, RNN, and Transformer were successful. The most challenging model to implement was the RNN, however, the use of the CNN pre-encoder along with the encoder-decoder GRU architecture resulted in the highest performance. The implementation of the GRU architecture for the RNN required careful planning and research to create a successful model and the use of gradient clipping solved the biggest problem of exploding gradients. Gradient clipping was also reused for the Transformer which provided the same benefit. In terms of performance, the RNN achieved the highest test accuracy performance as well as LA and forced CER when compared to the Transformer. The RNN also performed the best in terms of generalizability. Training time for the CNN was under twelve minutes and the training time for the RNN and Transformer was closer to thirty minutes. Caching the datasets consumed the most time, with the Transformer caching taking nearly ten hours. The Transformer was able to achieve training and validation accuracy performance greater than 96%, however, the model was prone to overfitting and poor generalization abilities as the LA accuracy and forced CER were extremely poor. Due to time constraints, this model could not be improved further and the poor generalization and overfitting is acknowledged.

### **Conclusion and Future Work**

Implementing the CNN, RNN, and Transformer models uncovered key insights such as the ability to powerful models that are not inherently large in size, but perform capably for the specified task. Across all three models, the benefit of pooling techniques like max pooling and adaptive pooling were extremely helpful in dimension reduction while retaining the most important information from the features extracted. Trial and error with hyperparameter tuning was a tedious task that required testing different combinations of search ranges and techniques to find the best performing setup for a model. In the future, expanding to an approach based on processing individual strokes with two dimensional convolution layers for the CNN would be interesting to see the benefits compared to the one dimensional convolution layers used here. Other alternative architectures like the LSTM and Transformer decoders with more attention heads and transformer blocks would also be interesting to explore. The limitations of the existing approaches include the use of pooling techniques which although provided the reduction in dimensions, it also came with the cost of the loss of spatial information from the strokes. Information loss was also present in the RNN as the context vector was a compressed fixed size. The addition of learning rate scheduling in training would be an additional benefit for all models that could be explored in the future.

## References

- K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- TorchVision contributors. Torchvision: Pytorch’s computer vision library. <https://github.com/pytorch/vision>, 2017.
- Casper da Costa-Luis et al. tqdm: A fast, extensible progress bar for python and cli, 2015. URL <https://github.com/tqdm/tqdm>.
- T. Ghosh, S. Sen, S. M. Obaidullah, K. C. Santosh, K. Roy, and U. Pal. Advances in online handwritten recognition in the last decades. *Computer Science Review*, 46:100515, 2022.
- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science and Engineering*, 9(3):90–95, 2007.
- Thomas Kluyver and contributors. h5py: A Python interface to the HDF5 data format. <https://github.com/h5py/h5py>, 2017.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Eli Stevens, Luca Antiga, and Thomas Viehmann. *Deep Learning with PyTorch*. Manning Publications, 2020.
- C. Y. Suen, C. Nadal, R. Legault, T. A. Mai, and L. Lam. Computer recognition of unconstrained handwritten numerals. *Proceedings of the IEEE*, 80(7):1162–1180, 1992.
- Hiroyuki Tanaka and contributors. editdistance: Fast implementation of the edit distance (levenshtein distance). <https://github.com/roy-ht/editdistance>, 2019.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Tyler Yep and contributors. torchinfo: Model summary in pytorch. <https://github.com/TylerYep/torchinfo>, 2020.